

AUTENTICACIÓN SQL Y WINDOWS:

DIFERENCIAS Y PRÁCTICAS SEGURAS

La autenticación asegura que solo usuarios válidos puedan acceder a los recursos del sistema.

SQL Server ofrece dos modos principales: **Autenticación SQL Server** y **Autenticación de Windows**. Elegir correctamente y configurarlos bien fortalece la seguridad de la base de datos.

AUTENTICACIÓN SQL SERVER

VS.

AUTENTICACIÓN DE WINDOWS



Usa cuentas propias de SQL Server para autenticar a los usuarios.

CARACTERÍSTICAS

- ★ Cuentas y contraseñas almacenadas en SQL Server.
- ★ No depende del dominio de Windows.
- ★ Útil para aplicaciones o accesos remotos sin Active Directory.
- ★ Mayor flexibilidad y control granulado.
- ★ Requiere gestión segura de contraseñas.

Usa las credenciales de Windows y el dominio (Active Directory) para autenticar a los usuarios.

CARACTERÍSTICAS

- ★ Usa cuentas de dominio o locales de Windows.
- ★ Centraliza la administración de usuarios.
- ★ Más segura (políticas de contraseña, bloqueos, auditoría por AD).
- ★ Ideal para entornos corporativos.
- ★ Requiere configuración de Kerberos/NTLM y permisos adecuados.



PRÁCTICAS SEGURAS PARA AMBOS MODOS



USA CONTRASEÑAS FUERTES

Mínimo 12 caracteres, combina mayúsculas, minúsculas, números y símbolos.



PRINCIPIO DE MÍNIMO PRIVILEGIO

Asigna solo los permisos necesarios para cada usuario o rol.



EVITA CUENTAS COMPARTIDAS

Cada usuario debe tener su propia cuenta para trazabilidad.



AUDITA Y MONITOREA

Habilita SQL Server Audit para registrar intentos de acceso y cambios de permisos.



MANTÉN ACTUALIZADO

Aplica parches y actualizaciones de SQL Server y Windows.



CIFRADO DE CONEXIONES

Usa TLS/SSL para proteger los datos en tránsito.

¿CUÁNDO USAR CADA UNO?

ESCENARIO	SQL SERVER AUTHENTICATION	WINDOWS AUTHENTICATION
Aplicaciones externas / internet	✓	✗
Entornos sin Active Directory	✓	✗
Entornos corporativos con AD	✗	✓
Integración con servicios de Windows	✗	✓
Necesidad de control granulado en BD	✓	✓

RECOMENDACIONES FINALES

- ★ Habilita solo el modo de autenticación que realmente necesites.
- ★ Si usas ambos modos, desactívalos y renómbrala y protégela.
- ★ Usa roles y grupos para simplificar la administración.
- ★ Revisa periódicamente los permisos y cuentas existentes.
- ★ Documenta y estandariza tus políticas de autenticación.

TIP RÁPIDO

En entornos corporativos, Windows Authentication es generalmente más segura y fácil de administrar.



COMANDOS ÚTILES (T-SQL)

```
-- Ver modo de autenticación actual
SELECT SERVERPROPERTY('IsIntegratedSecurityOnly');

-- Cambiar a modo mixto (SQL y Windows)
EXEC sp_configure 'login mode', 2; RECONFIGURE;
```



RECURSOS VISUALES



CUENTAS DE SERVICIO Y CONFIGURACIÓN DEL SERVIDOR

Las cuentas de servicio permiten que SQL Server y los componentes del sistema operen correctamente y de forma segura. Una correcta configuración del servidor reduce riesgos y mejora el rendimiento y la estabilidad. ❤️



¿QUÉ ES UNA CUENTA DE SERVICIO?

Es una cuenta de usuario utilizada por servicios de SQL Server y otros componentes para iniciar sesión y realizar tareas automatizadas en el servidor.



Mayor seguridad

Permite aplicar el principio de mínimo privilegio.



Mejor control

Facilita la administración y el monitoreo.



Estabilidad

Evita interrupciones por permisos incorrectos.

BUENAS PRÁCTICAS PARA CUENTAS DE SERVICIO

- ✓ Usar cuentas de dominio dedicadas, no cuentas personales.
- ✓ Aplicar el principio de mínimo privilegio: solo los permisos necesarios.
- ✓ Evitar cuentas locales siempre que sea posible.
- ✓ Configurar contraseñas seguras y con expiración controlada.
- ✓ Documentar el uso y propósito de cada cuenta.



TIPOS DE CUENTAS DE SERVICIO EN SQL SERVER

TIPO	DESCRIPCIÓN
 Cuenta de servicio del motor	Ejecuta el servicio del Motor de Base de Datos (SQL Server Database Engine). Ejemplo: SQLServiceAccount
 Cuenta de servicio del Agente SQL Server	Ejecuta el servicio del SQL Server Agent para programar tareas y alertas. Ejemplo: SQLAgentServiceAccount
 Cuenta de servicio de Reporting Services	Ejecuta los servicios de Reporting Services. Ejemplo: SQLRSServiceAccount
 Cuenta de servicio de Integration Services (SSIS)	Ejecuta los paquetes y servicios de SSIS. Ejemplo: SQLISServiceAccount

COMPARACIÓN

CUENTA DE SISTEMA

- Cuentas locales (ej. Local System).
- Fáciles de implementar.
- Menos seguras.
- Difíciles de administrar en múltiples servidores.



CUENTA DE DOMINIO

- Cuentas del dominio de la organización.
- Mayor seguridad y control.
- Permiten delegar permisos fácilmente.
- Recomendadas para ambientes productivos.



VS

CONFIGURACIÓN DEL SERVIDOR: ASPECTOS CLAVE



1. CONFIGURACIÓN DEL SERVICIO

- Asignar la cuenta de servicio correcta a cada servicio.
- Establecer el tipo de inicio: Automático (recomendado).
- Reiniciar el servicio después de cambios.



2. PERMISOS EN EL SISTEMA

- Asignar permisos NTFS a carpetas de datos, logs y backups.
- Permitir acceso de red si es necesario.



3. CONFIGURACIÓN DE MEMORIA

- Establecer memoria máxima y mínima del servidor.
- Evitar sobrecarga en el sistema operativo.
- Monitorear el uso de memoria.



4. CONFIGURACIÓN DE RED

- Configurar protocolos: TCP/IP habilitado.
- Definir puertos estáticos si es necesario.
- Restringir accesos a IP autorizadas.



5. CONFIGURACIÓN DE SEGURIDAD

- Habilitar cifrado (TDE).
- Revisar políticas de contraseñas.
- Configurar firewall de Windows.
- Mantener el servidor actualizado.

PASOS PARA CREAR Y ASIGNAR UNA CUENTA DE SERVICIO



1. CREAR CUENTA EN EL DOMINIO

Crear una cuenta dedicada en Active Directory.



2. ASIGNAR PERMISOS NECESARIOS

Otorgar permisos NTFS y permisos de SQL Server requeridos.



3. CONFIGURAR SERVICIO EN SQL SERVER

Desde SQL Server Configuration Manager, asignar la cuenta al servicio.



4. REINICIAR SERVICIO

Aplicar los cambios reiniciando el servicio correspondiente.



5. VERIFICAR FUNCIONAMIENTO

Probar conectividad, accesos y tareas programadas.

RECUERDA ❤️

Una correcta gestión de cuentas de servicio y una configuración adecuada del servidor son la base para mantener un entorno seguro, estable y eficiente.





CREACIÓN DE ROLES FIJOS Y PERSONALIZADOS

Los roles permiten agrupar permisos y simplificar la administración de la seguridad en SQL Server. Pueden ser roles fijos (ya definidos por el sistema) o roles personalizados (creados según las necesidades de la organización).

¿POR QUÉ USAR ROLES?

- ✓ Simplifican la administración de permisos.
- ✓ Mejoran la seguridad y el control de acceso.
- ✓ Facilitan la asignación de permisos a múltiples usuarios.



ROLES FIJOS (DEL SISTEMA)

Son roles predefinidos por SQL Server con permisos específicos. No se pueden modificar ni eliminar.

	ROL FIJO	DESCRIPCIÓN
	sysadmin	Acceso total al servidor. Puede realizar cualquier acción.
	serveradmin	Administra la configuración del servidor y los inicios de sesión.
	securityadmin	Administra logins y permisos del servidor.
	dbcreator	Puede crear, modificar y eliminar bases de datos.
	diskadmin	Administra archivos de disco en el servidor.
	processadmin	Puede ver y administrar procesos en ejecución.
	setupadmin	Administra los vínculos de SQL Server con otros servidores.
	bulkadmin	Realiza operaciones BULK INSERT.



ROLES PERSONALIZADOS (DE USUARIO)

Son roles creados por los administradores para adaptarse a los requerimientos específicos de la organización.

VENTAJAS



Agrupar permisos según funciones o departamentos.



Permiten un control más granular y personalizado.



Facilitan el cumplimiento de políticas internas.



Reducen el tiempo de administración.

EJEMPLOS DE ROLES PERSONALIZADOS

- Analistas: permisos de solo lectura en reportes.
- Operadores: pueden ejecutar procedimientos almacenados.
- SoporteTIC: acceso limitado para mantenimiento.
- LectoresBD: acceso de consulta a bases específicas.



¿CÓMO CREAR Y USAR ROLES PERSONALIZADOS?



1 CREAR EL ROL

Se crea el rol en la base de datos o en el servidor.

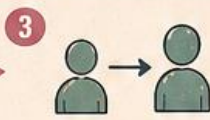
```
CREATE ROLE NombreRol;
```



2 ASIGNAR PERMISOS

Se otorgan los permisos necesarios al rol.

```
GRANT Permiso  
ON Objeto  
TO NombreRol;
```



3 AGREGAR USUARIOS AL ROL

Se agregan usuarios o logins existentes al rol.

```
ALTER ROLE NombreRol  
ADD MEMBER Usuario;
```



4 VERIFICAR PERMISOS

Se valida que el rol tenga los permisos correctos.

```
EXEC sp_helprolemember  
'NombreRol';
```



5 MANTENER Y AUDITAR

Revisar periódicamente los roles y permisos asignados.

Auditoría y monitoreo = Seguridad efectiva.

★ BUENAS PRÁCTICAS ★

- ✓ Usa roles en lugar de otorgar permisos directamente a usuarios.
- ✓ Asigna a los usuarios al rol adecuado, no más permisos de los necesarios.
- ✓ Revisa y actualiza los roles y sus permisos periódicamente.
- ✓ Documenta los roles y su propósito.
- ✓ Combina roles para crear estructuras de seguridad eficaces.

★ ESCENARIO DE EJEMPLO ★

NECESIDAD



El área de reportes necesita consultar información sin modificar datos.

SOLUCIÓN



Crear un rol ReporteLectura con permisos de SELECT.



Usuarios asignados al rol pueden ver la información sin riesgo de modificación.

COMANDOS ÚTILES (T-SQL)

- CREATE ROLE NombreRol;
- ALTER ROLE NombreRol ADD MEMBER Usuario;
- GRANT Permiso ON Objeto TO NombreRol;
- REVOKE Permiso ON Objeto FROM NombreRol;
- DROP ROLE NombreRol;



RECURSOS VISUALES



Roles



Permisos



Usuarios



Seguridad

CONTROL DE ACCESO

mediante

GRANT, DENY Y REVOKE



Controlar el acceso es asignar solo los permisos necesarios y retirar los innecesarios. SQL Server usa **GRANT, DENY y REVOKE** para gestionar permisos de forma precisa.



¿QUÉ SON?

GRANT, DENY y REVOKE son sentencias de Transact-SQL que permiten conceder, negar o quitar permisos sobre objetos y recursos de la base de datos, asegurando que cada usuario o rol tenga el nivel de acceso adecuado.

GRANT

Concede permisos a usuarios o roles para ejecutar acciones específicas sobre objetos.

Ejemplo:

```
GRANT SELECT, INSERT ON Ventas.Productos TO UsuarioVentas;
```

Permisos comunes:

- **SELECT** - Consultar datos
- **INSERT** - Insertar datos
- **UPDATE** - Actualizar datos
- **DELETE** - Eliminar datos
- **EXECUTE** - Ejecutar procedimientos
- **REFERENCES** - Crear claves foráneas

DENY

Niega permisos explícitamente, sobrescribiendo cualquier permiso concedido.

Ejemplo:

```
DENY DELETE ON Ventas.Productos TO UsuarioVentas;
```

¿Cuándo usarlo?

- Para bloquear acciones específicas.
- Tiene prioridad sobre GRANT.
- Útil en entornos con múltiples roles.

REVOKE

Quita permisos previamente concedidos o denegados.

Ejemplo:

```
REVOKE SELECT ON Ventas.Productos FROM UsuarioVentas;
```

Notas:

- Si se revoca un permiso concedido, se pierde el acceso.
- Si se revoca un DENY, el permiso puede volver a heredarse desde roles o niveles superiores.

ORDEN DE PRECEDENCIA



IMPORTANTE

Si un permiso es DENY, no se puede ejecutar, aunque haya sido GRANT desde otro rol o nivel.

NIVELES DONDE SE PUEDEN APLICAR PERMISOS



EJEMPLO COMPLETO

- | | |
|---|--|
| 1 Conceder permiso de lectura sobre una tabla | GRANT SELECT ON Ventas.Clientes TO Analista; |
| 2 Negar permiso de eliminación sobre la misma tabla | DENY DELETE ON Ventas.Clientes TO Analista; |
| 3 Revocar permiso de lectura cuando ya no se necesite | REVOKE SELECT ON Ventas.Clientes FROM Analista; |

BUENAS PRÁCTICAS

- ✓ Otorga solo los permisos mínimos necesarios (principio de mínimo privilegio).
- 👥 Usa roles para simplificar la administración.
- 📄 Revisa y audita permisos periódicamente.
- 🛡️ Evita usar DENY a menos que sea estrictamente necesario.
- 🔄 Documenta los permisos otorgados y sus justificaciones.

REFERENCIA RÁPIDA

COMANDO	ACCIÓN	SINTAXIS BÁSICA
GRANT	Concede permisos	GRANT permiso ON objeto TO usuario rol;
DENY	Niega permisos	DENY permiso ON objeto TO usuario rol;
REVOKE	Quita permisos	REVOKE permiso ON objeto FROM usuario rol;

RECUERDA

Un buen control de acceso protege la información, cumple políticas de seguridad y reduce riesgos.

¡Menos es más, pero seguro es mejor!



CIFRADO Y PROTECCIÓN DE DATOS (TDE, ALWAYS ENCRYPTED)

El cifrado protege la información confidencial para que solo usuarios autorizados puedan acceder a ella, incluso si los datos son interceptados o extraídos sin permiso.

¿POR QUÉ CIFRAR?

- Protege datos sensibles.
- Cumple normativas de seguridad y privacidad.
- Reduce el impacto ante robos o accesos no autorizados.
- Genera confianza en los usuarios y la organización.

TDE (TRANSPARENT DATA ENCRYPTION)

Cifra automáticamente los archivos de datos, registros de transacciones y backups a nivel de base de datos.



CARACTERÍSTICAS DE TDE

- Cifra a nivel de base de datos.
- Protege archivos de datos y logs.
- Utiliza un Certificado o una Clave Asimétrica.
- Es transparente para las aplicaciones.
- Protege backups y copias de base de datos.

ALWAYS ENCRYPTED

Cifra los datos sensibles a nivel de columna antes de que salgan de la aplicación al motor de base de datos.



CARACTERÍSTICAS DE ALWAYS ENCRYPTED

- Cifra a nivel de columna (datos sensibles).
- Las claves nunca salen del cliente.
- El servidor no puede ver los datos en texto plano.
- Requiere compatibilidad con drivers y aplicaciones.
- Ideal para proteger datos confidenciales en la nube.

TDE vs ALWAYS ENCRYPTED

ASPECTO	TDE	ALWAYS ENCRYPTED
Nivel de cifrado	Base de datos (archivos)	Columna (datos específicos)
Dónde ocurre el cifrado	En el servidor	En el cliente
Transparencia	Totalmente transparente	Requiere cambios en la aplicación
Protege de	Acceso físico al disco, robos, backups	DBA malintencionado, fugas internas, accesos no autorizados
Rendimiento	Bajo impacto	Mayor impacto (según diseño)
Requiere claves en cliente	No	Sí
Ideal para	Protección general de la BD	Datos muy sensibles y cumplimiento estricto

GESTIÓN DE CLAVES

TDE

- Crear Certificado o Clave Asimétrica en SQL Server.
- Crear Clave de Cifrado de Base de Datos.
- Habilitar TDE en la base de datos.
- Las claves se almacenan de forma segura en el servidor.

ALWAYS ENCRYPTED

- Crear Claves Maestras (en el cliente y/o en Azure Key Vault).
- Crear Claves de Columna para los datos sensibles.
- Configurar la aplicación para cifrar y descifrar los datos.
- Gestionar y rotar claves periódicamente.

BUENAS PRÁCTICAS

- Cifra siempre los datos sensibles.
- Usa TDE para protección a nivel de disco.
- Usa Always Encrypted para columnas críticas.
- Gestiona y rota claves periódicamente.
- Respalda y protege las claves de cifrado.
- Limita el acceso a certificados y claves.
- Monitorea accesos y auditorías de seguridad.

EJEMPLO: CIFRADO TDE

```
-- 1. CREAR CERTIFICADO
CREATE CERTIFICATE CertTDE
WITH SUBJECT = 'Certificado para TDE';

-- 2. CREAR CLAVE DE CIFRADO
CREATE DATABASE ENCRYPTION KEY
WITH ALGORITHM = AES_256
ENCRYPTION BY SERVER CERTIFICATE CertTDE;

-- 3. HABILITAR TDE
ALTER DATABASE MiBaseDeDatos
SET ENCRYPTION ON;
```

EJEMPLO: ALWAYS ENCRYPTED

```
-- 1. CREAR CLAVE MAESTRA EN CLIENTE
CREATE MASTER KEY ENCRYPTION BY
PASSWORD = 'ClaveSegura123!';

-- 2. CREAR CLAVE DE COLUMNA
CREATE COLUMN MASTER KEY ColKey
WITH (KEY_STORE_PROVIDER_NAME = N'MSSQL_CERTIFICATE_STORE',
KEY_PATH = N'CurrentUser/My/<thumbprint>');

-- 3. USAR EN LA TABLA
CREATE TABLE Clientes (
Id INT PRIMARY KEY,
Nombre NVARCHAR(100)
ENCRYPTED WITH (COLUMN_ENCRYPTION_KEY = ColKey,
ENCRYPTION_TYPE = DETERMINISTIC)
);
```

PROTEGER LA INFORMACIÓN
es proteger el futuro
de tu organización.



Seguridad hoy.
información
siempre.

AUDITORÍA Y MONITOREO

♥ de eventos con ♥

SQL SERVER AUDIT

Auditar te permite saber **QUÉ** pasó, **QUIÉN** lo hizo y **CUÁNDO**.
Así detectas riesgos, cumples normativas y fortaleces la seguridad de tu base de datos.

¿POR QUÉ AUDITAR?

- ✓ Detecta accesos no autorizados.
- ✓ Cumple políticas y normativas (ISO, SOX, etc.).
- ✓ Investiga incidentes de seguridad.
- ✓ Monitorea cambios y actividades críticas.
- ✓ Genera evidencia y trazabilidad.

¿QUÉ ES SQL SERVER AUDIT?

Es una característica nativa de SQL Server que permite registrar eventos del servidor y de la base de datos en uno o más destinos de auditoría.

COMPONENTES PRINCIPALES



AUDIT

Contenedor que agrupa las especificaciones de auditoría y el destino donde se almacenan los eventos.



AUDIT SPECIFICATION

Define QUÉ eventos se van a auditar y SOBRE QUÉ (servidor o base de datos).



AUDIT DESTINATION

Define DÓNDE se guardarán los eventos (archivo, Windows Security Log, Application Log).

EVENTOS QUE PUEDES AUDITAR



Inicio de sesión exitoso o fallido



Uso de permisos altos (GRANT, DENY, REVOKE)



Cambios de permisos y roles



Fallos de acceso y errores



Consultas SELECT, INSERT, UPDATE, DELETE



Cambios en la esquema y DDL



Acceso y cambios a objetos



Operaciones de BACKUP y RESTORE



Ejecución de procedimientos almacenados



Intentos de inyección o actividad sospechosa

DESTINOS DE AUDITORÍA



ARCHIVO (Recomendado)

Guarda los eventos en archivos .sqlaudit que pueden revisarse con SQL Server Management Studio o herramientas externas.



WINDOWS SECURITY LOG

Envía los eventos al registro de seguridad de Windows.



WINDOWS APPLICATION LOG

Envía los eventos al registro de aplicaciones de Windows.



El destino más común y recomendado es ARCHIVO, por su flexibilidad y facilidad de análisis.

¿CÓMO FUNCIONA? (FLUJO GENERAL)



USUARIO O PROCESO
Realiza una acción en SQL Server.



EVENTO GENERADO
SQL Server detecta el evento según la auditoría configurada.



CAPTURA DEL EVENTO
La especificación de auditoría filtra y captura el evento seleccionado.



ALMACENAMIENTO
El evento se escribe en el destino de auditoría configurado.



ANÁLISIS Y MONITOREO
Se revisan los eventos para detectar anomalías y tomar acciones.

HERRAMIENTAS ÚTILES

- SQL Server Management Studio (Audit Reports)
- Extended Events (para análisis avanzado)
- Power BI / Excel (para reportes y dashboards)
- Scripts T-SQL personalizados

EJEMPLO RÁPIDO (T-SQL)

1 CREAR AUDIT

```
CREATE SERVER AUDIT SeguridadAudit TO FILE (FILEPATH = 'D:\Auditoria\');
```

2 CREAR ESPECIFICACIÓN DE AUDITORÍA

```
CREATE SERVER AUDIT SPECIFICATION EspecLogin FOR SERVER AUDIT SeguridadAudit ADD (FAILED_LOGIN_GROUP), ADD (SUCCESSFUL_LOGIN_GROUP);
```

3 HABILITAR AUDITORÍA

```
ALTER SERVER AUDIT SeguridadAudit WITH (STATE = ON);
```

4 HABILITAR ESPECIFICACIÓN

```
ALTER SERVER AUDIT SPECIFICATION EspecLogin WITH (STATE = ON);
```

5 CONSULTAR EVENTOS

```
SELECT event_time, action_id, session_server_principal_name, succeeded, additional_information FROM sys.fn_get_audit_data('D:\Auditoria\');
```

Ajunta la ruta del archivo según tu servidor.

BUENAS PRÁCTICAS

- ✓ Audita solo lo necesario para no afectar el rendimiento.
- ✓ Revisa y rota los archivos de auditoría periódicamente.
- ✓ Protege los archivos de auditoría (con permisos y cifrado).
- ✓ Monitorea el tamaño de los archivos y el impacto en disco.
- ✓ Documenta las políticas de auditoría y revisalas regularmente.
- ✓ Combina auditoría con alertas y notificaciones.

REPORTE E INTERPRETACIÓN

event_time	action_id	session_server_principal_name	succeeded	additional_information
2034-05-28 10:15:22	LGIF	usuario1	0	Login failed
2034-05-28 10:16:05	LGIS	usuario1	1	Login succeeded
2034-05-28 10:18:30	SC	usuario1	1	SELECT * FROM Clientes

CÓDIGOS COMUNES (action_id)

LGIF = Login failed
LGIS = Login succeeded
DP = Database permission change
SC = Schema change
UP = User permission change

CLAVES PARA EL ANÁLISIS

- ¿Quién hizo la acción?
- ¿Qué acción realizó?
- ¿Cuándo ocurrió?
- ¿Fue exitosa o fallida?
- ¿Qué objeto o recurso afectó?